MATH423: Introduction to Computational Topology

Mehmetcik Pamuk (mpamuk@metu.edu.tr)
Department of Mathematics, Middle East Technical University

2026-27 FALL

This section is written for students who like to program. It contains three case studies, worked out in detail with real numbers, and a set of programming exercises that turn the definitions of Weeks 1–3 into working code. All the numbers quoted below were computed with the small Python toolkit of Section 4.4, which uses only the standard library.

4.1 Case Study: The Stanford Bunny

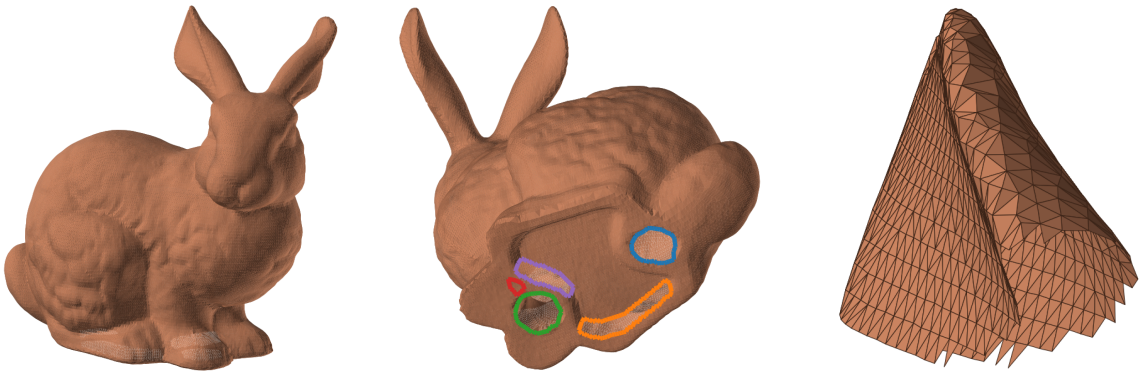
A short history

In 1993–94 Greg Turk, then a postdoctoral researcher in Marc Levoy's group at Stanford, was developing methods for turning laser scans of real objects into triangle meshes. He needed a test object, and around Easter he bought a terra cotta rabbit, about 19 cm tall, at a home and garden shop on University Avenue near the Stanford campus. Terra cotta reflects red laser light well and has a dull surface, which made it ideal for the scanner.

The rabbit was scanned with a Cyberware 3030 MS laser range scanner. Each scan sees the object from one direction only, so 10 scans were taken from different directions. Each scan produces a *range image*, a grid of measured points with depths. The scans were then aligned and merged into a single triangle mesh by a method Turk and Levoy called *zippering*, published at SIGGRAPH 1994. The result, with 35,947 vertices and 69,451 triangles, was made freely available through the Stanford 3D Scanning Repository.

The bunny quickly became the standard test object of computer graphics and geometry processing. A new algorithm for mesh simplification, compression, smoothing, texture mapping, fracture simulation or rendering of fur was typically demonstrated on the bunny, so that results from different research groups could be compared directly. Before it, the standard test object was the Utah teapot, modelled by hand by Martin Newell in 1975 from mathematical curves. The bunny was different in an important way: it came from *measured data*, with all the defects of real measurements. It is still used today, and later scans from the same repository, such as the Dragon and the Happy Buddha with over a million triangles each, played the same role for larger meshes.

For us the bunny is interesting because its defects are topological; see Figure 16. According to Turk, the mesh has five holes. Two are real: the clay figurine is hollow, and its base has two openings. The other three are places the scanner could not see, one under the chin and two on the base. A mesh with holes is not a closed surface, and many algorithms (computing volume, 3D printing, deciding inside from outside) fail on it. Later methods, such as the volumetric method of Curless and Levoy (1996), were designed to produce closed surfaces from scans.



(a) the scanned bunny (b) seen from below (c) the tip of an ear

Figure 16: The Stanford bunny, rendered directly from the original mesh with 69,451 triangles. (a) The whole model. (b) The model seen from below, with its five boundary loops (the holes) outlined in different colours; the outlines are drawn on top of the surface, so they remain visible even where the surface would hide them. (c) A close-up of 1,345 triangles near the tip of an ear: at this scale the surface is visibly a simplicial complex.

Reading the bunny topologically

Running the tools of Week 2 on the original mesh file gives the following.

Vertices listed in the file	35,947
Vertices used by at least one triangle	34,834
Edges	104,288
Triangles	69,451
Edges lying in exactly two triangles	104,065
Edges lying in exactly one triangle (boundary edges)	223
Edges lying in three or more triangles	0
Connected components	1
Boundary loops	5, of lengths $22, 39, 40, 42, 80$
Consistently oriented	yes

Several lessons can be read off from this table.

- **Check the counts.** With boundary edges, the relation $3F = 2E$ of Week 2 becomes $3F = 2E - B$, where B is the number of boundary edges: each triangle has three edges, interior edges are counted twice and boundary edges once. Indeed $3 \cdot 69{,}451 = 208{,}353 = 2 \cdot 104{,}288 - 223$.

- **Unused vertices.** The file lists 1,113 vertices that belong to no triangle, leftovers of the merging process. A program that takes V from the header of the file gets $\chi = 35{,}947 - 104{,}288 + 69{,}451 = 1{,}110$, which is nonsense for a connected surface. Using only the vertices that actually occur gives

$$\chi = 34{,}834 - 104{,}288 + 69{,}451 = -3.$$

- **Genus.** For a compact connected orientable surface of genus g with b boundary circles, $\chi = 2 - 2g - b$, since each boundary circle bounds a removed disk and removing a disk lowers χ by 1 (Week 2). Here $-3 = 2 - 2g - 5$ gives $g = 0$. So the bunny is a sphere with five holes, as it should be: its ears do not form handles.

- **Filling the holes.** If each boundary loop is capped off by adding one new vertex and joining it to all edges of the loop, the number of vertices increases by 5, while the numbers of edges and triangles both increase by 223. The new Euler characteristic is $-3 + 5 = 2$: a closed sphere, which could now be 3D printed.

Each of these facts is obtained by a single pass over the list of triangles. Nothing about the shape of the bunny was used, only its combinatorics.

4.2 Case Study: Protein Pockets and Docking

The problem

A protein is a long chain of amino acids that folds into a specific three-dimensional shape, containing thousands of atoms. Many drugs are small molecules, called *ligands*, that work by fitting into a cavity or groove on the surface of a target protein and blocking its function, an idea going back to Emil Fischer's *lock and key* picture of 1894. **Docking** is the computational problem of predicting whether and how a ligand binds to a protein. Docking programs, such as the widely used AutoDock Vina, search over the possible positions and orientations of the ligand and score each one by an approximate binding energy. Before any of this can be done, one must know *where* the possible binding sites are, and how large they are. This is a question about the shape of empty space around the protein, and topology answers it.

The three-dimensional structures of more than two hundred thousand proteins, determined experimentally, are freely available in the Protein Data Bank (PDB). A structure file lists, for each atom, its chemical element and the coordinates of its center, in ångströms ($1\,\text{Å} = 10^{-10}\,\text{m}$).

The geometric model

Each atom is modelled as a ball whose radius is its van der Waals radius, about 1.7 Å for carbon, 1.55 Å for nitrogen and 1.52 Å for oxygen. A solvent molecule or a ligand is modelled as a ball of radius ρ, called the **probe**; for water, $\rho \approx 1.4$ Å. The center of the probe can be at a point p only if the probe does not overlap any atom, that is, if

$$d(p, x_i) \geq r_i + \rho \quad \text{for every atom center } x_i \text{ with radius } r_i.$$

So the space available to the probe center is the complement of the union of the *inflated* balls of radii $r_i + \rho$. This is exactly the situation of Week 3: a union of balls, whose topology is captured by the Čech complex, or in practice by the much smaller alpha complex.

- A **cavity** (or void) is a bounded component of the free space: a region inside the protein completely enclosed by atoms. In three dimensions it is detected by 2-dimensional homology, which counts enclosed voids.

- A **pocket** is a depression in the surface with an opening, its *mouth*, to the outside. It is not a hole for small probes, but it becomes a cavity once the probe is too large to pass through the mouth.

A worked two-dimensional example

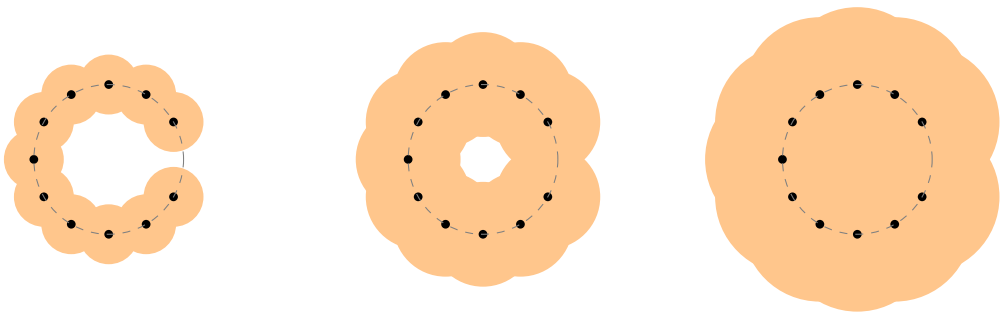
To see how this works, consider a toy protein in the plane: 11 atoms of radius $r_a = 1.7$ Å, with centers on a circle of radius 6 Å at the angles $30°, 60°, \ldots, 330°$. The atom at angle $0°$ is missing, which creates a mouth. In the plane, holes in a union of disks are counted by 1-dimensional homology, but the picture is the same as in three dimensions. Write $R = r_a + \rho$ for the inflated radius. Two quantities control everything.

- Consecutive atoms are $2 \cdot 6 \sin 15° \approx 3.11$ Å apart, so their inflated disks overlap as soon as $R \geq 1.55$ Å. Since already $r_a = 1.7 > 1.55$, the wall of the pocket is closed even for $\rho = 0$.
- The two atoms at $30°$ and $330°$, on either side of the mouth, are $2 \cdot 6 \sin 30° = 6$ Å apart. Their inflated disks meet when $R = 3$ Å.

As R increases, three things happen (Figure 17).

1. For $R < 3$, the mouth is open. The free space inside the pocket is connected to the outside, so there is no hole: the pocket is accessible.
2. At $R = 3$ the mouth closes and a hole is *born*. For $3 \leq R < 6$ the free space has a bounded component: a probe of this size fits inside the pocket, but it cannot enter or leave through the mouth.
3. The center of the circle is at distance 6 Å from every atom, and it is the point inside the ring farthest from all atom centers. So at $R = 6$ the hole is filled; it *dies*.

The hole therefore exists for $R \in [3, 6)$, that is, for probe radii $\rho = R - 1.7$ in the interval $[1.3, 4.3)$ Å. Read chemically: a water molecule ($\rho = 1.4$) is just too large to pass through the mouth, while ligands of effective radius up to 4.3 Å fit inside the pocket. This interval of ρ is what persistent homology (Weeks 7–8) records as a *bar* in a barcode. A long bar indicates a large, deep pocket, a candidate binding site; small dents in the surface of the protein produce short bars that die soon after they are born.



$R = 2.4$: mouth open $R = 4.2$: enclosed cavity $R = 6.2$: filled

Figure 17: The toy protein: 11 atom centers (black dots) on a circle of radius 6 Å, with a missing atom on the right forming the mouth of a pocket. Shaded disks have the inflated radius $R = r_a + \rho$. The free space for the probe center (unshaded) has a hole exactly for $3 \leq R < 6$.

From the toy to real proteins

For real proteins the same idea is carried out in three dimensions with atoms of different radii, using *weighted* alpha complexes, which handle balls of different radii. Liang, Edelsbrunner and Woodward used alpha shapes to measure pockets and cavities in proteins, Edelsbrunner, Facello and Liang gave a precise definition of pockets with mouths, and the CASTp web server, based on this work, computes them for any structure in the PDB. More recently, persistent homology of protein–ligand complexes has been used to build features for machine learning models predicting binding strength, for example in the TopologyNet models of Cang and Wei. Topology does not replace chemistry: binding also depends on charges, hydrogen bonds and the flexibility of the protein. What topology provides is a reliable, fast and coordinate-free first step: a list of the places where a ligand could go, together with their size.

4.3 Case Study: Placing and Checking Sensors

Section 4 described how a network of sensors can certify its own coverage. Here we look at a concrete version of the problem, including the question of where to put the sensors in the first place. Each sensor monitors a disk of radius r_c and can communicate with sensors within distance r_b.

Where to put the sensors

Suppose sensors can be placed at will and we want to cover a large region with as few sensors as possible. Two natural patterns are the square grid and the triangular lattice; see Figure 18.

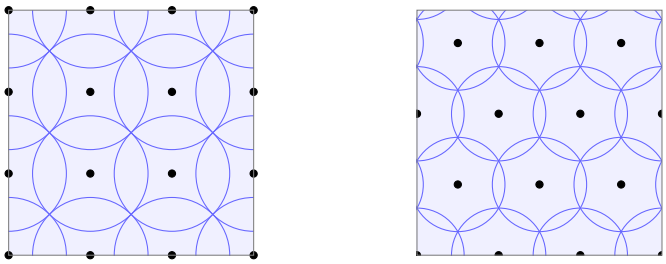
- **Square grid** with spacing a. The point farthest from all sensors is the center of a square, at distance $a/\sqrt{2}$. So we need $a \le \sqrt{2}\, r_c$. Each sensor accounts for one square of area a^2, so the number of sensors per unit area is at least $1/(2r_c^2) = 0.5/r_c^2$.
- **Triangular lattice** with spacing a. The point farthest from all sensors is the center of an equilateral triangle, at distance $a/\sqrt{3}$, its circumradius. So we need $a \le \sqrt{3}\, r_c$. Each sensor accounts for a rhombus of area $\frac{\sqrt{3}}{2}a^2$, so the number of sensors per unit area is $\frac{2}{\sqrt{3}\,a^2} = \frac{2}{3\sqrt{3}\,r_c^2} \approx 0.385/r_c^2$.

The triangular lattice needs about 23% fewer sensors. Kershner proved in 1939 that it is in fact the best possible: no arrangement of equal disks covers the plane more economically.

Notice the role of $\sqrt{3}$. If the communication radius satisfies $a \le r_b < \sqrt{3}\, a$, each sensor communicates exactly with its six nearest neighbors, and the Rips complex of the network is the triangulation of the plane by equilateral triangles of side a. The optimal pattern $a = \sqrt{3}\, r_c$ with $r_b = a$ then satisfies $r_c = r_b/\sqrt{3}$: it is precisely the extremal case of the covering proposition of Section 4, in which every Rips triangle is covered with nothing to spare.

Checking a network without knowing positions

Once the sensors are deployed, some may fail, and their exact positions may be unknown. The network only knows which pairs of working sensors can communicate. Consider a 4×4 square grid of sensors with spacing 1, and let b_1 denote the number of independent



square grid, $r_c = a/\sqrt{2}$ triangular lattice, $r_c = a/\sqrt{3}$

Figure 18: Sensors with the same spacing a, each drawn with the smallest coverage disk for which the pattern covers the plane. The triangular lattice needs only $r_c = a/\sqrt{3}$ instead of $a/\sqrt{2}$, so for a fixed r_c it can use a larger spacing and fewer sensors.

holes in the Rips complex. (This is the first Betti number, defined in Week 5; the toolkit below computes it.)

Network	Radius	b_1 of Rips
full grid	$r_b = 1$	9
full grid	$r_b = \sqrt{2}$	0
interior sensor $(1,1)$ failed	$r_b = \sqrt{2}$	1
interior sensors $(1,1)$ and $(2,2)$ failed	$r_b = \sqrt{2}$	2
interior sensors $(1,1)$ and $(1,2)$ failed	$r_b = \sqrt{2}$	1
corner sensor $(0,0)$ failed	$r_b = \sqrt{2}$	0

Several features of the table are worth understanding.

- With $r_b = 1$ the sensors only talk along the grid lines, and each of the 9 unit squares is a hollow 4-cycle. These holes are artefacts of the communication radius: the squares may well be covered. The Rips complex can only certify coverage when r_b is large enough compared with the spacing.
- With $r_b = \sqrt{2}$ the diagonals appear and each unit square becomes a full 3-simplex, so the complex is contractible. The proposition of Section 4 then certifies coverage as long as $r_c \geq \sqrt{2}/\sqrt{3} \approx 0.82$.
- When the interior sensor $(1,1)$ fails, its four nearest neighbors form a 4-cycle with no simplex filling it, and $b_1 = 1$. The hole is real: the point $(1,1)$ is at distance 1 from every remaining sensor, so it is uncovered whenever $r_c < 1$.
- Two failed sensors that are diagonal neighbors produce two separate holes, while two failed sensors that are adjacent produce one larger hole. The number b_1 counts holes, not failed sensors.
- When the corner sensor fails, the corner $(0,0)$ is no longer covered if $r_c < 1$, but $b_1 = 0$: no hole appears, because the missing region is on the outside of the network. Coverage near the boundary cannot be detected from the communication graph alone. This is why de Silva and Ghrist assume that the sensors near the boundary of the region (the *fence*) know that they are on the boundary, and use homology *relative* to the fence.

4.4 A Toolkit

The following module, `simplicial.py`, is distributed with these notes. It is deliberately short, so that every line can be understood. The function `betti_numbers` computes the numbers of independent holes in each dimension by Gaussian elimination over $\mathbb{Z}/2$; how and why it works is the subject of Weeks 5 and 6, and until then it can be used as a black box.

```
from itertools import combinations
from collections import Counter
from math import dist

def closure(maximal):
    """All non-empty faces of the given simplices, as sorted tuples."""
    K = set()
    for s in maximal:
        s = tuple(sorted(s))
        for k in range(1, len(s) + 1):
            K.update(combinations(s, k))
    return K

def f_vector(K):
    c = Counter(len(s) - 1 for s in K)
    return [c[d] for d in range(max(c) + 1)]

def euler_characteristic(K):
    return sum((-1) ** (len(s) - 1) for s in K)

def star(K, sigma):
    sigma = set(sigma)
    return {t for t in K if sigma <= set(t)}

def closed_star(K, sigma):
    return closure(star(K, sigma))

def link(K, sigma):
    sigma = set(sigma)
    return {t for t in closed_star(K, sigma) if not sigma & set(t)}

def rips(points, r, max_dim=2):
    """Simplices whose points are pairwise within distance r."""
    n = len(points)
    nbr = {i: set() for i in range(n)}
    for i, j in combinations(range(n), 2):
        if dist(points[i], points[j]) <= r:
            nbr[i].add(j); nbr[j].add(i)
    K = {(i,) for i in range(n)}
    layer = [(i,) for i in range(n)]
    for _ in range(max_dim):
        new = []
        for s in layer:
            common = set.intersection(*(nbr[v] for v in s))
```

```
            new += [s + (w,) for w in common if w > s[-1]]
        K.update(new)
        layer = new
    return K

def betti_numbers(K):
    """Betti numbers over Z/2 (black box until Week 5)."""
    dim = max(len(s) for s in K) - 1
    index = {d: {} for d in range(dim + 1)}
    for s in sorted(K):
        index[len(s) - 1][s] = len(index[len(s) - 1])
    rank = {}
    for d in range(1, dim + 1):
        pivots, r = {}, 0
        for s in index[d]:
            col = 0
            for face in combinations(s, d):
                col ^= 1 << index[d - 1][face]
            while col:
                low = col.bit_length() - 1
                if low in pivots:
                    col ^= pivots[low]
                else:
                    pivots[low] = col; r += 1
                    break
        rank[d] = r
    return [len(index[d]) - rank.get(d, 0) - rank.get(d + 1, 0)
            for d in range(dim + 1)]
```

Note that `rips` uses the actual distance threshold r (an edge whenever $d(x_i, x_j) \le r$), which is the convention natural for sensor networks; in the notation VR_ϵ of Week 3 this is $\epsilon = r/2$.

Programming Exercises

Answers that you can use to test your code are given in brackets.

1. **Warm-up.** Use the toolkit on the boundary of the tetrahedron, given by its maximal simplices $\{1,2,3\}, \{1,2,4\}, \{1,3,4\}, \{2,3,4\}$. Compute the f-vector, χ, and the star, closed star and link of the vertex 1 and of the edge $\{1,2\}$, and compare with the examples of Week 3. [$f = (4,6,4)$, $\chi = 2$; the star of 1 has 7 simplices, its closed star 13, its link 6; the link of $\{1,2\}$ is $\{\{3\},\{4\}\}$.] Then write a function `link_condition(K, a, b)` and check the two examples of Section 4 (octahedron and tetrahedron).

2. **A surface recognizer.** Write a program that takes a list of triangles and decides: (i) whether it is a closed surface, by checking that each edge lies in exactly two triangles and that the link of each vertex is a single cycle; (ii) whether it is connected; (iii) whether it is orientable, using the breadth-first orientation algorithm of Week 2; and (iv) which surface it is, using χ and the classification theorem. Test it on the 9-vertex torus, on the Klein bottle produced by the code below, and on the Möbius

band $\{1,2,3\},\{2,3,4\},\{3,4,5\},\{4,5,1\},\{5,1,2\}$, for which step (i) should report 5 boundary edges.

```
def grid_surface(n=3, twist=False):
    """Triangulated square with opposite sides identified:
    the torus, or the Klein bottle if twist=True."""
    def label(x, y):
        if twist and y == n:
            x, y = -x, 0
        return n * (y % n) + (x % n)
    T = []
    for y in range(n):
        for x in range(n):
            a, b = label(x, y), label(x + 1, y)
            c, d = label(x + 1, y + 1), label(x, y + 1)
            T += [(a, b, c), (a, c, d)]
    return T
```

[Torus: $\chi = 0$, orientable. Klein bottle: $\chi = 0$, non-orientable.]

3. **Betti numbers cannot see orientation.** Compute `betti_numbers` for the torus and the Klein bottle of the previous exercise, and for the Möbius band. [Torus and Klein bottle: $(1,2,1)$; Möbius band: $(1,1,0)$.] Over $\mathbb{Z}/2$ the torus and the Klein bottle look the same, so the orientation test of Exercise 2 is really needed. (In Week 5 we will see that homology with integer coefficients does distinguish them.)

4. **Stanford bunny forensics.** Download the bunny (the file `bun_zipper.ply` in the archive on the Stanford 3D Scanning Repository, or a copy converted to OBJ format from one of the many collections of standard test models). Write a reader for the file format and reproduce the table in the bunny case study: the numbers of used and unused vertices, edges and triangles, the edges in one, two, or three or more triangles, the connected components, the boundary loops and their lengths, the orientation check, χ and the genus. Explain why the formula $3F = 2E - B$ holds.

5. **Repairing the bunny.** Cap each boundary loop of the bunny with a new vertex, as described in the case study, orienting the new triangles consistently with their neighbors. Check that the result is a closed orientable surface with $\chi = 2$. Write the repaired mesh to an OBJ file and look at it in a mesh viewer; where are the holes?

6. $\star$ **Topology-preserving simplification.** Repeatedly contract the shortest edge of the bunny whose two endpoints are interior vertices and which satisfies the link condition of Section 4, updating the list of triangles after each contraction. Stop at about 5,000 triangles. Check that $\chi = -3$ and the five boundary loops are preserved, and compare with what happens if you ignore the link condition. (Efficient implementations use a priority queue for the edge lengths.)

7. **Sensor grids.** Reproduce the table of the sensor case study using `rips` and `betti_numbers`, with `max_dim=3`. Then do the same for a 5×5 patch of the triangular lattice with spacing 1 and $r_b = 1$, before and after removing the central sensor.

[Before: $b_1 = 0$; after: $b_1 = 1$.] Finally, for each experiment, estimate the uncovered area directly by sampling many random points of the square and checking their distance to the nearest working sensor, with $r_c = r_b/\sqrt{3}$, and compare with the topological answer.

8. $\star$ **Random deployment.** Drop N sensors uniformly at random in the unit square, add a fence of sensors spaced $r_b/2$ apart along the boundary, and compute b_1 of the Rips complex at radius r_b. For fixed r_b and increasing N, estimate the probability that $b_1 = 0$, and the probability that the square is actually covered by disks of radius $r_b/\sqrt{3}$. Plot both against N. Which is larger, and why can the topological test never report coverage when there is a hole in the interior?

9. **The toy protein.** Represent the free space of the toy protein by a fine grid of points in the square $[-14, 14]^2$, and for each inflated radius R mark the points at distance greater than R from all atom centers. Count the bounded components of the marked points by flood fill (breadth-first search on the grid). Plot the number of holes against R and check that the hole is born at $R = 3$ and dies at $R = 6$. Warning: near the values of R where two disks just overlap, single grid points get trapped in the narrow cusps between two circles, and your program will report many tiny holes. Explain why, and decide how to filter them out. Then move the atom at 30° to the angle 20°; how do the birth and death values change?

10. $\star$ **A real protein.** Download a protein structure from the Protein Data Bank (`rcsb.org`) in PDB format. The lines starting with `ATOM` contain the coordinates in columns 31–54 and the chemical element in columns 77–78. Using van der Waals radii (1.70 for C, 1.55 for N, 1.52 for O, 1.80 for S, in Å), a probe radius ρ, and a three-dimensional grid with spacing 0.5 Å, find the free space and count the enclosed cavities by a flood fill started at the boundary of the grid. Repeat for ρ between 0 and 3 Å. Which cavities persist over a long range of ρ?

11. **Data without coordinates.** Let X be the set of binary strings of length 3, with the Hamming distance (the number of positions in which two strings differ). Compute `betti_numbers(rips(X, 1, 2))` using a Hamming distance function in place of `dist`. [$(1, 5)$.] Explain why there are no triangles, and why the answer is $E - V + 1$ for the graph of the cube. Find a formula for strings of length n. [$n2^{n-1} - 2^n + 1$.]

12. **Sperner's lemma in code.** Subdivide the triangle with corners $(0, 0)$, $(1, 0)$, $(0, 1)$ into n^2 small triangles: cut each square of the $n \times n$ grid of mesh $1/n$ along its diagonal parallel to the line $x + y = 1$, and keep the triangles lying in the big triangle. Give the vertices random colors satisfying the Sperner condition of Section 4. Count the fully colored triangles and check that the number is odd. Then implement the door-walking algorithm and verify that it always ends in a fully colored triangle.

4.5 Matroids: Greedy Algorithms and Simplicial Complexes

Every computer science student meets Kruskal's algorithm for minimum spanning trees: sort the edges by length, and add each edge unless it creates a cycle. It is a greedy algorithm, which never reconsiders a decision, and yet it always finds the optimal tree.

Most greedy algorithms are not this lucky. The theory of **matroids**, introduced by Whitney in 1935, explains exactly when greedy algorithms work, and it does so through a family of simplicial complexes. It also explains why the algorithm computing Betti numbers in our toolkit is a greedy algorithm in disguise.

Definition and first examples

Whitney noticed that two kinds of independence behave in the same way: linearly independent sets of vectors, and sets of edges of a graph containing no cycle (forests). He extracted the common properties.

Definition 4.1. A **matroid** M on a finite set E is a non-empty family $\mathcal{I}$ of subsets of E, called **independent sets**, such that:

1. **(Hereditary property)** If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$.
2. **(Exchange property)** If $A, B \in \mathcal{I}$ and $|A| < |B|$, then there is an element $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$.

A maximal independent set is a **basis**, and a minimal dependent set (a subset not in $\mathcal{I}$, all of whose proper subsets are in $\mathcal{I}$) is a **circuit**.

The exchange property implies that all bases have the same size: if B_1 and B_2 are bases with $|B_1| < |B_2|$, then B_1 could be enlarged by an element of B_2, contradicting maximality. This common size is the **rank** of M.

Compare the hereditary property with the definition of an abstract simplicial complex in Week 3. If every single element of E is independent, then the non-empty independent sets form an abstract simplicial complex on the vertex set E, the **independence complex** $IN(M)$. Its simplices are the independent sets, its facets are the bases, and its dimension is $\text{rank}(M) - 1$. A matroid is thus a simplicial complex with an extra property, the exchange property.

Example 4.2.

- **Linear matroids.** Let E be the set of columns of a matrix over a field, and call a set of columns independent if it is linearly independent. The hereditary property is clear, and the exchange property is the Steinitz exchange lemma of linear algebra: if $|A| < |B|$, the vectors of B span a space of dimension larger than that spanned by A, so some $x \in B$ lies outside the span of A. The rank of the matroid is the rank of the matrix.

- **Graphic matroids.** Let E be the set of edges of a graph G, and call a set of edges independent if it contains no cycle. The circuits are the cycles of G, and if G is connected the bases are its spanning trees and the rank is $|V(G)| - 1$.

- **Uniform matroids.** The matroid $U_{k,n}$ on n elements whose independent sets are all subsets of size at most k. Its independence complex is the $(k-1)$-skeleton of the full simplex on n vertices.

The first two examples are closer than they look. Write the **incidence matrix** of a graph over $\mathbb{Z}/2$: one row for each vertex, one column for each edge, with a 1 in the rows of the two endpoints of the edge. A set of columns is linearly dependent over $\mathbb{Z}/2$ exactly when some non-empty subset of them sums to zero, that is, when some non-empty set of the edges meets every vertex an even number of times. Such an edge set contains a

cycle, and every cycle is such a set. So the graphic matroid of G is the linear matroid of its incidence matrix. In Week 5 this matrix will reappear as the *boundary matrix* ∂_1 of G, viewed as a 1-dimensional simplicial complex.

Example 4.3. Let $G = K_4$, the complete graph on four vertices, with 6 edges. Any one or two edges form a forest. A set of three edges is a forest unless it is one of the 4 triangles of K_4, so there are $\binom{6}{3} - 4 = 16$ forests with three edges; these are the spanning trees of K_4 (Cayley's formula gives $4^{4-2} = 16$). No set of four edges is a forest. So $IN(M(K_4))$ is a 2-dimensional complex with f-vector $(6, 15, 16)$ and

$$\chi = 6 - 15 + 16 = 7.$$

Example 4.4. Not every simplicial complex comes from a matroid. Consider the complex with facets $\{1, 2\}$ and $\{3\}$. Taking $A = \{3\}$ and $B = \{1, 2\}$, neither $\{1, 3\}$ nor $\{2, 3\}$ is a simplex, so the exchange property fails. A similar failure occurs for **matchings** in a graph, sets of edges no two of which share a vertex. In the path $a - b - c - d$, the matchings $A = \{bc\}$ and $B = \{ab, cd\}$ satisfy $|A| < |B|$, but neither $\{bc, ab\}$ nor $\{bc, cd\}$ is a matching. The matchings form a simplicial complex, but not a matroid.

There is a clean topological way of recognizing matroids among simplicial complexes. For a set W of vertices, the **induced subcomplex** $K[W]$ consists of all simplices of K with all vertices in W. A complex is **pure** if all its facets have the same dimension. It is a standard fact that a simplicial complex is the independence complex of a matroid if and only if every induced subcomplex is pure; this is the exchange property rephrased. In the first non-example above, K itself is not pure.

The greedy algorithm

Suppose each element $e \in E$ has a weight $w(e)$, and we want a basis of maximal total weight. The greedy algorithm sorts E by decreasing weight and goes through the list once, adding each element to the current set if the result is still independent.

Theorem 4.5 (Rado, Edmonds)**.** *Let $\mathcal{I}$ be a non-empty family of subsets of a finite set E satisfying the hereditary property. Then the greedy algorithm finds a maximal independent set of maximal total weight for every choice of weights if and only if $\mathcal{I}$ is a matroid.*

Proof that greedy works for matroids. Let $g_1, \ldots, g_r$ be the elements chosen by the greedy algorithm, in the order chosen, so that $w(g_1) \geq \cdots \geq w(g_r)$. They form a basis: if some independent set were larger, the exchange property would give an element that greedy should have added. Let $b_1, \ldots, b_r$ be any other basis, ordered so that $w(b_1) \geq \cdots \geq w(b_r)$. We show that $w(g_i) \geq w(b_i)$ for every i, which gives the theorem by summing. Suppose not, and let i be the first index with $w(g_i) < w(b_i)$. Apply the exchange property to $A = \{g_1, \ldots, g_{i-1}\}$ and $B = \{b_1, \ldots, b_i\}$: there is $b_j \in B \setminus A$ with $A \cup \{b_j\}$ independent, and $w(b_j) \geq w(b_i) > w(g_i)$. So greedy examined b_j before g_i. At that moment its current set was contained in A, so adding b_j to it would have kept it independent by the hereditary property, and greedy would have chosen b_j. Then $b_j \in A$, a contradiction. □

For the graphic matroid with the weights $w(e) = -\text{length}(e)$, the greedy algorithm is exactly Kruskal's algorithm, and the theorem explains why it always produces a minimum spanning tree. For matchings it fails: in the path $a - b - c - d$ with weights $w(ab) = w(cd) = 2$ and $w(bc) = 3$, greedy takes bc and stops with weight 3, while $\{ab, cd\}$ has weight 4. Maximum weight matchings can still be found efficiently, but by more sophisticated algorithms (Edmonds' blossom algorithm).

Matroids and persistence

Programming Exercise 4 of Section 4 asked you to show that the values of ϵ at which components of VR_ϵ merge are half the edge lengths of a minimum spanning tree. Matroids explain why. As ϵ grows, the edges of the Rips complex appear in order of length. An edge merges two components exactly when it creates no cycle with the earlier edges, that is, exactly when greedy (Kruskal) would accept it.

The same principle governs persistent homology in every dimension. In Weeks 6–8 we will reduce the boundary matrix column by column, from left to right in the order in which simplices appear, adding earlier columns to later ones. A column becomes zero exactly when it is a linear combination of earlier columns. So the columns that survive the reduction are the ones the greedy algorithm would choose in the *linear* matroid of the columns: the reduction algorithm *is* the greedy algorithm for that matroid. For a filtered graph the columns are those of the incidence matrix, and the surviving columns are exactly the edges chosen by Kruskal's algorithm. The function `betti_numbers` in our toolkit runs this greedy procedure to compute ranks, and then uses $b_k = n_k - \operatorname{rank}\partial_k - \operatorname{rank}\partial_{k+1}$, where n_k is the number of k-simplices.

The topology of matroid complexes

Independence complexes of matroids have very special topology. Björner showed that if M has rank r, then $IN(M)$ is *shellable*, a strong combinatorial condition which implies that $IN(M)$ is homotopy equivalent to a wedge of $(r-1)$-dimensional spheres; the number of spheres can be zero. Since a wedge of m spheres of dimension $r-1$ has Euler characteristic $1+(-1)^{r-1}m$, the number m can be read off from χ.

Example 4.6.

- For the triangle graph K_3, the forests are all sets of at most 2 edges, so $IN(M(K_3))$ is the hollow triangle, a single circle.
- For K_4 the rank is 3 and $\chi = 7 = 1+6$, so $IN(M(K_4))$ is homotopy equivalent to a wedge of 6 two-dimensional spheres. The toolkit confirms this: `betti_numbers` returns $(1, 0, 6)$.
- For the uniform matroid $U_{n,n}$ every set is independent, IN is a full simplex, and the number of spheres is 0.

Matroid Exercises

13. **Recognizing matroids.** Write a function `is_matroid(family)` that checks the hereditary and exchange properties by brute force, and returns a violating pair (A, B) when the exchange property fails. Test it on the forests of K_4, on $U_{2,4}$, on the matchings of the path $a-b-c-d$, and on the complex with facets $\{1,2\}$ and $\{3,4\}$. [Matroid, matroid, not a matroid, not a matroid; for the last one, try $A=\{2\}$ and $B=\{3,4\}$.] Also write a function that checks whether every induced subcomplex is pure, and confirm that it gives the same answers.

14. **Independence complexes of complete graphs.** Build $IN(M(K_n))$ for $n = 3, 4, 5$ and compute its f-vector and Betti numbers with the toolkit. [K_3: $(3,3)$ and $(1,1)$. K_4: $(6,15,16)$ and $(1,0,6)$. K_5: $(10,45,110,125)$ and $(1,0,0,51)$.] Check that in each case the complex has homology only in the top dimension, as Björner's

theorem predicts, and that the last entry of the f-vector agrees with Cayley's formula n^{n-2}.

15. **Greedy, with and without a matroid.** Implement the greedy algorithm with an *independence oracle*, a function that tells whether a set is independent. Use it with a forest test, implemented by union-find, to obtain Kruskal's algorithm, and compare its output with a brute-force search over all spanning trees on random weighted copies of K_5. Then use it with a matching test on random weighted small graphs, and find, by random search, weights for which greedy is not optimal.

16. **Forests are independent columns.** For K_4, check that a set of edges is a forest if and only if the corresponding columns of the incidence matrix are linearly independent over $\mathbb{Z}/2$. (Represent each column as a Python integer with two bits set, and use the elimination loop of `betti_numbers`.) Then take random points in the plane, order all the edges of the complete graph by length, reduce the incidence matrix column by column in this order, and check that the surviving columns are exactly the edges chosen by Kruskal's algorithm.

17. $\star$ **The rank function.** The rank of a subset $A \subseteq E$ is the size of a largest independent subset of A. For the graphic matroid of K_5, compute ranks and test the *submodular inequality* $r(A \cup B) + r(A \cap B) \leq r(A) + r(B)$ on many random pairs of edge sets. Explain what $|A| - r(A)$ counts for a set of edges A, and relate it to the number of independent cycles of the graph formed by A from Section 4.

Further Reading

- H. M. Berman et al., The Protein Data Bank, *Nucleic Acids Research* 28 (2000).
- A. Björner, The homology and shellability of matroids and geometric lattices, in *Matroid Applications* (N. White, ed.), Cambridge University Press, 1992.
- Z. Cang, G.-W. Wei, TopologyNet: topology based deep convolutional and multi-task neural networks for biomolecular property predictions, *PLoS Computational Biology* 13 (2017).
- B. Curless, M. Levoy, A volumetric method for building complex models from range images, *Proc. SIGGRAPH* (1996).
- H. Edelsbrunner, M. Facello, J. Liang, On the definition and the construction of pockets in macromolecules, *Discrete Applied Mathematics* 88 (1998).
- J. Edmonds, Matroids and the greedy algorithm, *Mathematical Programming* 1 (1971).
- R. Kershner, The number of circles covering a set, *American J. Mathematics* 61 (1939).
- J. B. Kruskal, On the shortest spanning subtree of a graph and the traveling salesman problem, *Proc. Amer. Math. Soc.* 7 (1956).
- B. Lee, F. M. Richards, The interpretation of protein structures: estimation of static accessibility, *J. Molecular Biology* 55 (1971).
- J. Oxley, *Matroid Theory*, 2nd edition, Oxford University Press, 2011.
- R. Rado, Note on independence functions, *Proc. London Math. Soc.* (3) 7 (1957).

- O. Trott, A. J. Olson, AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading, *J. Computational Chemistry* 31 (2010).

- G. Turk, M. Levoy, Zippered polygon meshes from range images, *Proc. SIGGRAPH* (1994).

- G. Turk, The Stanford Bunny, `faculty.cc.gatech.edu/~turk/bunny/bunny.html`.

- H. Whitney, On the abstract properties of linear dependence, *American J. Mathematics* 57 (1935).

- The Stanford 3D Scanning Repository, `graphics.stanford.edu/data/3Dscanrep`.